# LLM Driven Reinforcement Learning for ARC

Yifan Jing(ivaning@umich.edu) Zhuoyu Chen(zhuoyuc@umich.edu)
Zikai Zhou(zikaiz@umich.edu) Danqi Hu(danqihu@umich.edu)
Haolun Cai(haolunca@umich.edu)

March 13, 2026

**Abstract**

Please write an abstract (executive summary) here in a paragraph.
1 paragraph:

1. Why is the problem important and interesting?
2. What are things that are held back because your problem has not been solved yet?
3. What are great things that could happen if you solved your problem?

## 1 Introduction

1. Problem description and motivation.
2. Why do you want to solve this problem?
3. What's the impact if you can solve this problem?

Large Language Models (LLMs) have demonstrated remarkable capabilities in code generation tasks. However, they still struggle with **complex reasoning and self-correction** when generating code for novel, abstract problems. A critical limitation is that current LLMs often produce incorrect code on the first attempt and lack the ability to iteratively refine their solutions based on execution feedback.

The **Abstraction and Reasoning Corpus (ARC)** provides an ideal testbed for this challenge. ARC tasks require models to infer transformation rules from a few input-output grid examples and generate code that generalizes to unseen test inputs. Unlike conventional coding benchmarks (e.g., HumanEval, MBPP) that test memorization of common programming patterns, ARC evaluates **core reasoning and generalization abilities**—skills that remain elusive for state-of-the-art LLMs.

**The central problem we address is:**

> *How can we enable LLMs to learn from execution feedback and iteratively improve their code generation through reinforcement learning, particularly for tasks requiring abstract reasoning?*

Specifically, we aim to answer the following research questions:

1. **Can reinforcement learning improve code generation accuracy** beyond supervised fine-tuning alone, by rewarding successful execution outcomes?

2. **Can a self-correction mechanism**—where models revise their code based on error messages and incorrect outputs—be effectively learned through RL?

3. **What reward shaping strategies** are most effective for guiding LLMs toward correct solutions in sparse-reward environments where most generated code fails?

This problem is significant because solving it would bridge the gap between current LLM capabilities and human-like iterative problem-solving, with broad applications in automated programming, scientific discovery, and AI-assisted reasoning.

### 1.1 Significance

ARC is designed to test whether an AI system can reason abstractly and generalize from a few examples. That is a skill that human find natural but modern machine learning struggles with. Even the most powerful large language model perform poorly on ARC problems. The tasks require discovering previse transformation rather than recognizing statistical patterns. This gap highlights the problem with modern AI systems: they lack reliable mechanisms for symbolic reasoning, self-correction, and learning from small amounts of data.

Due to the reason that this problem remains unsolved, progress is limited in many areas that depend on structured reasoning. For example, program synthsis, automated debugging, and decision-making in complex environment. Systems that rely only on pattern matching often fail when exact logic, compositional reasoning, or correctness guarantees are required.

This project addresses current limitations by integrating code generation with execution-based feedback and reinforcement learning. Rather than directly predicting answers, the model is trained to write and iteratively refine programs based on their performance. If effective, this methodology may enable AI systems to reason more robustly, adapt with minimal data, and offer more interpretable solutions for ARC and a broad spectrum of real-world reasoning challenges.

### 1.2 Evaluation

The proposed approach is evaluated on the **Abstraction and Reasoning Corpus (ARC)**, a benchmark designed to assess abstract reasoning and generalization from a small number of examples. For each ARC task, the model is provided with the training input-output grid pairs and is required to generate a Python program that implements the inferred transformation. The generated program is executed on the training examples, and execution feedback, including runtime errors and incorrect outputs, is incorporated to enable iterative refinement. After a limited number of correction steps, the final program is evaluated on the held-out test input. Following the official ARC evaluation protocol, a task is considered successfully solved only if the predicted output grid exactly matches the ground-truth output.

The primary evaluation metric is the task success rate, defined as the proportion of ARC tasks solved correctly on the test inputs. In addition, the effectiveness of learning from execution feedback is analyzed by measuring the number of refinement iterations required before a correct program is produced and the frequency with which generated programs execute without errors. Performance is compared before and after reinforcement learning, as well as against supervised fine-tuning without iterative correction, in order to isolate the impact of reinforcement learning and execution-based feedback on reasoning and generalization.

## 2 Methodology

1. How are you going to solve this problem?

2. Why should the proposed method work?

3. Provide explanations/rationale of why you chose this specific method.

4. Provide technical details of your approach if you are proposing a novel method.

5. Description of the pipeline. Including a figure(e.g., block diagram) that explains the pieline will be very helpful.

Compared to PPO, GRPO removes the need for a learned value function and instead uses group-relative reward normalization. This provides a stable learning signal even when absolute rewards are sparse, making GRPO particularly suitable for ARC-style program synthesis where most sampled programs fail.

We perform post-training of an instruction-following code LLM using Group Relative Policy Optimization (GRPO). Our objective is to train the model to generate executable programs for ARC tasks and to improve solutions through interaction-based feedback and self-correction. [**?** ]

Each ARC task provides training input-output grid pairs $\{(x_i, y_i)\}_{i=1}^{n}$ and a test input $x_{\text{test}}$. The model generates a Python program $c$ mapping $x \mapsto \hat{y}$. We execute $c$ on the training inputs to obtain outputs $\{\hat{y}_i\}$ and diagnostic feedback $\mathcal{F}$. The model may revise its code for up to $T$ iterations. The entire multi-step generation process is treated as a trajectory and optimized with GRPO.

Let $\pi_\theta$ denote the autoregressive policy. For prompt $p$, the model produces a trajectory $\tau$ (including code, feedback, and revisions):

$$\tau \sim \pi_\theta(\cdot \mid p), \qquad \log \pi_\theta(\tau \mid p) = \sum_{t=1}^{|\tau|} \log \pi_\theta(a_t \mid s_t).$$

After execution, we assign a reward

$$R(\tau) = R_{\text{train}}(c) + \lambda_{\text{fmt}} R_{\text{fmt}}(\tau) - \lambda_{\text{err}} R_{\text{err}}(c),$$

where $R_{\text{train}}$ measures correctness on training pairs, $R_{\text{fmt}}$ encourages well-formed executable code, and $R_{\text{err}}$ penalizes runtime errors or timeouts. A sparse version of correctness is

$$R_{\text{train}}(c) = \mathbb{1}\big[\forall i,\ \hat{y}_i = y_i\big].$$

For each prompt, we sample $K$ trajectories $\{\tau^{(k)}\}_{k=1}^{K}$ from $\pi_{\theta_{\text{old}}}$, compute rewards $\{R^{(k)}\}$, and form group-relative advantages:

$$\mu_R = \frac{1}{K}\sum_{k=1}^{K} R^{(k)}, \quad \sigma_R = \sqrt{\frac{1}{K}\sum_{k=1}^{K}(R^{(k)} - \mu_R)^2 + \epsilon}, \quad A^{(k)} = \frac{R^{(k)} - \mu_R}{\sigma_R}.$$

We then optimize a clipped PPO-style objective

$$\mathcal{L}_{\text{GRPO}}(\theta) = \mathbb{E}_p\left[\frac{1}{K}\sum_{k=1}^{K} \min\big(r_\theta(\tau^{(k)})A^{(k)}, \text{clip}(r_\theta(\tau^{(k)}), 1-\varepsilon, 1+\varepsilon)A^{(k)}\big)\right],$$

with $r_\theta(\tau) = \pi_\theta(\tau \mid p)/\pi_{\theta_{\text{old}}}(\tau \mid p)$, plus a KL penalty toward a reference model $\pi_{\text{ref}}$:

$$\mathcal{L}(\theta) = \mathcal{L}_{\text{GRPO}}(\theta) - \beta\, \mathbb{E}[\text{KL}(\pi_\theta \| \pi_{\text{ref}})].$$

This procedure encourages trajectories that produce executable and correct programs with fewer correction steps.

# 3 Related work

1. What are existing methods?
2. What are the state-of-the-art methods for this problem?
3. How is your approach similar to other existing work?
4. How is your approach different from the related work?

The ARC benchmark aims to measure the ability to infer rules from a small number of examples (train data) and transfer them to new problems (test data), rather than achieving a high score by fitting a large amount of data on a single task. **?** ] argues that the performance of a machine learning model on a single fixed task is strongly influenced by prior knowledge and training scale, so that ARC uses grid transformations and few-shot supervision data with prior knowledge to highlight differences in generalization. The ARC official competition guide further sets up the data and evaluation process, such as each task is provided in JSON format, a small set of training input-output pairs, and several test inputs, and the goal of ARC is how to infer the rule from train data and generate outputs from test data[**?** ].

In the direction of formulating the ARC problem as a reinforcement learning (RL) problem, ARCLE provides a dedicated RL environment from ARC, because directly applying RL on grids faces a large action space, sparse rewards hard to reach, and also diverse task types. **?** ] shows that the conventional RL methods, such as proximal policy optimization (PPO), can learn on individual tasks in this framework.

Besides, some work recognizes ARC solving as analogical reasoning because the model has the ability to extract transferable knowledge from known train data, and to apply the knowledge to similar but unseen tasks. **?** ] compares DreamerV3 (world-model) with PPO (model-free) on ARC tasks, and reports advantages of model-based methods for single-task learning and transfer across similar tasks. The benefit of this method is that it can align well with RL tools, but the art of designing action and reward function can be a very difficult problem for the policy model to learn program-like rules in the ARC benchmark.

# 4 Experimental results

(Preliminary) experimental results:

1. Milestones achieved so far(add all relevant experimental results).
2. How do these results support your claim?

## 4.1 Baseline Result

A baseline LLM-based ARC solver is first evaluated before the application of reinforcement learning. The system prompts an LLM to generate Python programs that implement transformation rules inferred from the training grid pairs. Given the training input-output examples of each ARC task, the model generates a Python function that maps an input grid to the corresponding output grid. The generated program is then executed on the training examples to verify correctness.

If the generated program fails on the training inputs due to incorrect outputs or runtime errors, the model receives execution feedback and is permitted to generate a revised solution. The final program is evaluated on the held-out test input according to the official ARC evaluation protocol.

We evaluate two models. First, **Qwen3-VL-30B-Instruct**, a vision-language model, is evaluated on the ARC-AGI-1 benchmark with deterministic decoding (temperature = 0.0, max retries = 2). Second, **Qwen3.5-35B-A3B-GPTQ-Int4**, a mixture-of-experts model (35B total /

3B active parameters, 4-bit quantized), is evaluated on the ARC-AGI-2 training set with stochastic decoding (temperature = 0.7, max retries = 5). Qwen3.5 is the open-source model we plan to fine-tune with RL. Note that ARC-AGI-2 is approximately a superset of ARC-AGI-1, retaining 96.6% of the original tasks and adding 347 new ones.

#### 4.1.1 Task Success Rate

The primary evaluation metric is task success rate, defined as the proportion of ARC tasks whose predicted output grid exactly matches the ground-truth test output.

| Model | Dataset | Split | Solved | Total | Rate |
|---|---|---|---|---|---|
| Qwen3-VL-30B | ARC-AGI-1 | Training | 45 | 400 | 11.2% |
| Qwen3-VL-30B | ARC-AGI-1 | Evaluation | 19 | 400 | 4.9% |
| Qwen3.5-35B-A3B | ARC-AGI-2 | Training$^\dagger$ | 121 | 175 | 69.1% |

Table 1: Task success rate on ARC benchmarks. Qwen3-VL uses deterministic decoding (temp=0.0, 2 retries). Qwen3.5 uses stochastic decoding (temp=0.7, 5 retries). $^\dagger$Partial evaluation; 175 of 1,000 ARC-AGI-2 training tasks completed so far. The two models are evaluated on different benchmarks and settings, so their results are not directly comparable.

The Qwen3.5-35B-A3B model achieves a 69.1% success rate on the 175 ARC-AGI-2 tasks evaluated so far (pass@5 with temp=0.7). More importantly, its pass@1 rate of 58.3% indicates that the reward signal is not too sparse—a majority of sampled programs succeed on the first attempt, providing ample positive reward signal for GRPO training.

#### 4.1.2 Retry Effectiveness

The system enables the model to revise generated code based on execution feedback. Table 2 shows the pass@K results for Qwen3.5 on ARC-AGI-2, where each additional attempt provides diminishing but meaningful gains.

| K | Pass@K | Marginal Gain |
|---|---|---|
| 1 | 58.3% | — |
| 2 | 63.4% | +5.1% |
| 3 | 66.9% | +3.4% |
| 4 | 68.6% | +1.7% |
| 5 | 69.1% | +0.6% |

Table 2: Pass@K for Qwen3.5-35B-A3B on ARC-AGI-2 (175 tasks, temp=0.7). Each retry provides diminishing but non-trivial improvement.

The high pass@1 rate (58.3%) confirms that the base model already solves a majority of tasks on the first attempt, while retries with execution feedback push the cumulative rate to 69.1%. This pattern—strong initial performance with diminishing marginal returns—suggests the model benefits from both stochastic sampling diversity and feedback-driven correction. Figure 1 visualizes the pass@K curve alongside the Qwen3-VL baseline for reference.

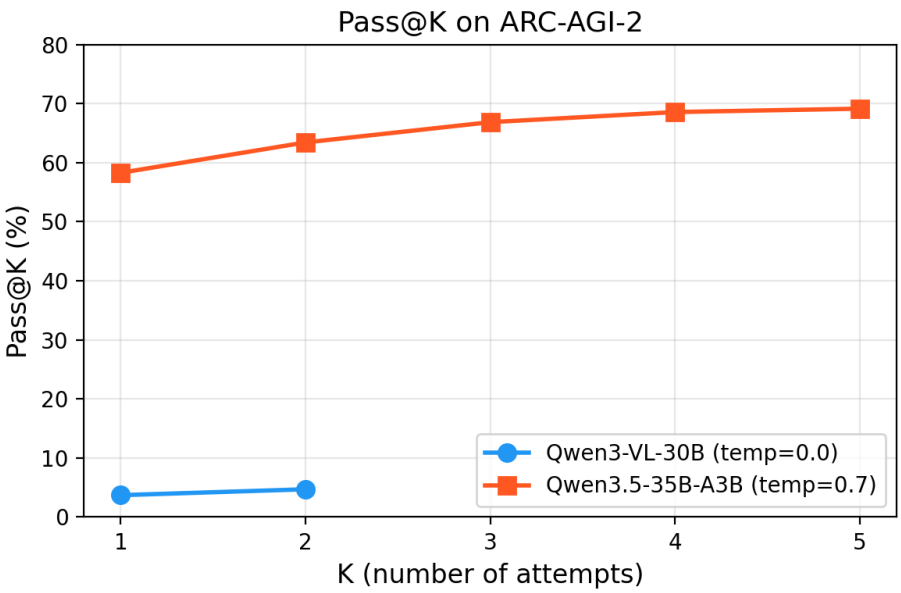


Figure 1: Pass@K curves on ARC-AGI-2. Qwen3-VL results (on ARC-AGI-1) are shown for reference only and are not directly comparable due to different benchmarks and settings.

#### 4.1.3 Prompt Token Analysis

The relationship between prompt length and task success is further analyzed. Tasks with shorter prompts exhibit higher success rates than tasks with longer prompts (Figure 2). However, this correlation should be interpreted with caution: longer prompts correspond to tasks with larger grids, which are inherently more difficult. The observed relationship is therefore likely a confounding effect of task complexity rather than a direct causal relationship between prompt length and model performance.

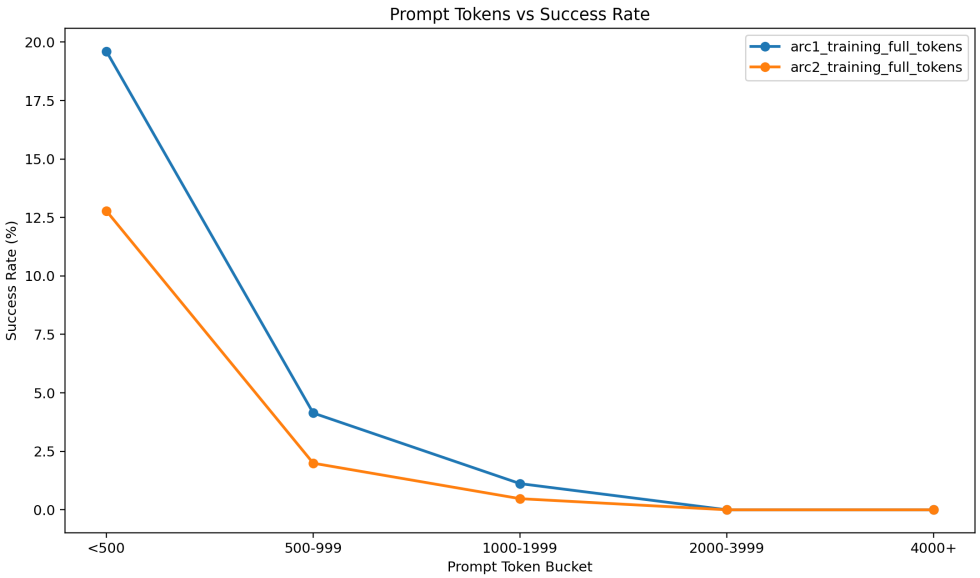


Figure 2: Distribution of prompt token counts across ARC tasks for ARC-AGI-1 and ARC-AGI-2. ARC-AGI-2 tasks generally require longer prompts due to increased task complexity.

#### 4.1.4 Failure Mode Analysis

Failure cases are categorized according to execution outcome. For Qwen3.5, out of 433 LLM calls across 175 tasks, 264 (61%) result in training failure (the program does not reproduce the training examples), 40 (9%) produce wrong output on test inputs despite passing training, and 129 (30%) are solved correctly (Figure 3).

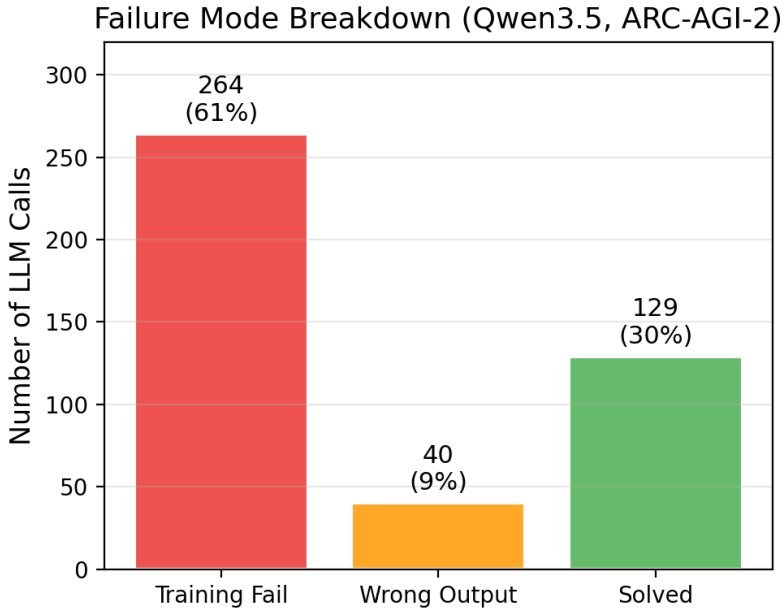


Figure 3: Failure mode breakdown for Qwen3.5 on ARC-AGI-2 (433 LLM calls, 175 tasks). Training failure dominates, indicating that inferring the correct rule remains the primary challenge.

Training failure remains the dominant failure mode, suggesting that the model frequently fails to infer the correct transformation rule from the few-shot examples. Notably, in the wrong-output cases, cell-level accuracy is high (mean 89.8%, median 97.8%), indicating that the model often captures most of the transformation rule but makes errors on edge cases or specific grid cells. This near-miss behavior suggests that partial credit reward shaping could provide useful learning signal for RL training.

#### 4.1.5 How These Results Support Our Claim

The experimental results yield several key observations that support the feasibility of our proposed RL approach.

Most importantly, the 58.3% pass@1 rate of Qwen3.5-35B-A3B on ARC-AGI-2 demonstrates that the reward signal is not too sparse for RL training—a majority of sampled programs receive positive reward on the first attempt, which is essential for GRPO to produce meaningful policy gradients. This is a critical prerequisite: if the base model rarely produced correct solutions, the RL reward signal would be too sparse to learn from.

Retry analysis shows diminishing but meaningful returns from additional attempts, indicating that the model can benefit from execution feedback but does not yet have an effective mechanism to systematically leverage it. Failure mode analysis reveals that training failure (failing to reproduce training examples) remains the dominant failure mode at 61% of LLM calls, confirming that inferring the correct transformation rule is the primary challenge. The high cell accuracy in wrong-output cases (median 97.8%) further suggests that partial credit reward shaping could provide additional gradient signal beyond binary correctness.

These findings motivate the proposed reinforcement learning framework, which utilizes execution outcomes as reward signals to improve the model's capacity for rule inference and iterative solution refinement.

## 5 Future milestones

1. Dates and sub-goals (please set sub-goals on a weekly basis so that they can be done in a week).

March 16 - March 29:
Build the GRPO training pipeline for ARC, including code generation, code execution, reward calculation, and feedback collection. Run small-scale experiments to check whether training is stable.

March 30 - April 12:
Tune the reward function and training settings, and run training on a larger set of ARC tasks to improve model performance.

April 13 - April 30:
Evaluate the trained model on ARC-AGI-1 and ARC-AGI-2, analyze the final results, and prepare the final report and presentation.

## 6 Conclusion

1. Summary of your progress and your final expected goal (what do you expect to achieve or demonstrate for the final project?).

In this project, we explore how to improve LLM-based code generation for ARC using reinforcement learning with execution feedback. ARC is difficult because it requires the model to infer abstract rules from only a few examples, which is still challenging for current LLMs. Our method uses GRPO to train the model based on whether the generated code is correct, executable, and able to improve through feedback.

So far, we have finished the baseline experiments and identified the main problem: prompting alone is not enough for strong reasoning and self-correction on ARC. Our early experiments with Qwen3.5-35B-A3B show that the reward is not too sparse—the model achieves 58.3% pass@1 on ARC-AGI-2 tasks, providing a substantial fraction of positive reward signals for GRPO training. Furthermore, the high cell accuracy in near-miss cases (median 97.8%) suggests that partial credit reward shaping could provide additional gradient signal. For the final project, we hope to show that this training method can improve the model's pass@1 accuracy and help it generate more correct programs for ARC tasks.

## 7 Author Contributions

1. Please see the guideline "Contribution of each team member" below.